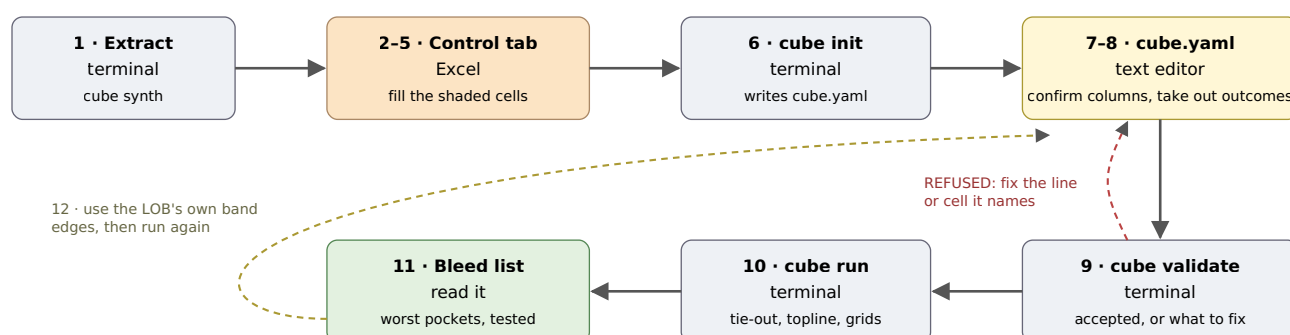


Origination analysis: finding where a book loses more than its share

Who this is for: the analyst preparing an origination analysis for a bank's line of business (LOB). **What you finish with:** a ranked list of the pockets of the book that lose more than their share, in dollars, with a test on each one. **Walked:** 25 September 2026, on commit 360a71e, on a synthetic book of 2,000 loans. No real bank data was used.

The cube file format (steps 7 and 8) was being reworked on the day of this walk. If your `cube.yaml` has a `columns: block` with `means: lines` instead of `key:`, `booked:`, `outcome: lines`, this part of the procedure is out of date. Walk it again and replace steps 7 and 8.



The route. Four kinds of screen: the terminal, the Control tab in Excel, the cube file in a text editor, and the run's output. Numbers are the steps below.

Before you start

- A terminal open in the origination-cube folder, with the tool installed (`pip install -e .`). After that, the command is `cube`.
- Excel, to fill in the Control tab.
- The loan extract as a CSV. It needs a loan number, the booked amount, a yes/no outcome, GCO dollars and RANR dollars. The run stops without any of them.

Every command below is typed exactly as shown. Where it says `demo/loans.csv`, use your `extract`'s path.

Step 1: Get the extract

For practice, make the synthetic book:

```
cube synth --out demo --rows 2000
```

```
terminal - origination-cube
$ cube synth --out demo --rows 2000
wrote demo/loans.csv and demo/cube.yaml
(exit code 0)
```

Right: it says wrote demo/loans.csv and demo/cube.yaml. The synthetic book hides one bad pocket: scores under 620 that came through the Broker channel. You should find it at the end.

Use demo/loans.csv, not demo/cube.yaml. The cube file the demo writes already has the judgment calls filled in for you. Make your own in steps 2 to 8.

Step 2: Write the Control tab

```
cube control --out control.xlsx
```

```
terminal - origination-cube
$ cube control --out control.xlsx
wrote control.xlsx
(exit code 0)
```

Right: wrote control.xlsx.

Step 3: Open the Control tab in Excel

Open control.xlsx. The shaded cells (ringed in red) are the ones still waiting for an answer. There are eight on a new tab.

Control center					
Fill in the shaded cells before running. Those are the calls we make on every job: what's material, how many loans is enough to matter, and how much worse than its peers a pocket has to be. Everything else starts on a reasonable setting, so change it only if the population calls for it. Pick from the list, or type your own number in the next column and that wins. If a row says re-run, run the script again after changing it.					
Shaded = still needs an answer	Choose	Or enter your own use	What it means	Takes effect	key
What the cube looks at	Every column alone, and every band crossed with every dimension (recommended)	n/a	pairs	re-run	grids
Which cuts to build	One level - cut each flagged pocket by every other column (recommended)	n/a	1 Shows which third factor explains the bleed inside the pocket. Most of the new pockets will be small.	re-run	drill_depth
How far to drill into a flagged pocket			Needs an answer before we run. Pick one, or enter your own.	re-run	min_age_months
How old a loan must be to count					
How columns are recognised	12 (recommended)		12 Term (36, 48, 60) or a grade from 1 to 10 is read as categories, not cut into bands.	re-run	few_values
A number column with this many values or fewer is a category	50 (recommended)		50 State (50 or so) is still a dimension; a city or a dealer name is raised as a question instead.	re-run	many_values
A text column with more values than this is too fine to cut by	5 (recommended)		5 Fine enough to see a trend, with each band holding a fifth of the loans.	re-run	band_count
How many bands a number column is cut into	Each band holds about the same number of loans (recommended)	n/a	equal_loans	re-run	band_cut
Where the band edges fall					
Is it enough loans to matter			Needs an answer before we run. Pick one, or enter your own.	live	min_loans
Fewest loans in a pocket before it is tested			Needs an answer before we run. Pick one, or enter your own.	live	min_events
Fewest loans with a loss before a loss rate is tested					
Is it material			Needs an answer before we run. Pick one, or enter your own.	live	materiality
Smallest excess loss worth reporting					
Does it underperform			Needs an answer before we run. Pick one, or enter your own.	live	compare_to
What a pocket is judged against			Needs an answer before we run. Pick one, or enter your own.	live	worse_at
How much worse than its comparison a pocket must be			Needs an answer before we run. Pick one, or enter your own.	live	better_at
How much better than its comparison a pocket must be					
Is it real			Needs an answer before we run. Pick one, or enter your own.	live	confidence
How sure a difference must be before it counts					
How often a real gap should be caught	80% (recommended)		0.8 The usual standard. Used only to say how big a gap each pocket's size could show; it never flags anything.	live	power
Allowing for testing many pockets at once	Hold down the share of lucky finds (Benjamini-Hochberg) (recommended)	n/a	bh	live	many_tests
Of the pockets that pass, about 5% at 95% are expected to be luck. Suits scanning for leads.					
Proving it					
Where a flagged pocket is tested	Found on earlier originations, confirmed on later ones (recommended)	n/a	time	re-run	proof
The strongest evidence for the LOB: the pattern held on loans it was not found in. Needs enough seasoned recent loans.					

Close up, the shaded rows ask what's enough loans, what's material, and how much worse counts as worse:

Is it enough loans to matter		
Fewest loans in a pocket before it is tested		Needs an answer before we run. Pick one, or enter your own.
Fewest loans with a loss before a loss rate is tested		Needs an answer before we run. Pick one, or enter your own.
Is it material		
Smallest excess loss worth reporting		Needs an answer before we run. Pick one, or enter your own.
Does it underperform		
What a pocket is judged against	n/a	Needs an answer before we run. Pick one, or enter your own.
How much worse than its comparison a pocket must be		Needs an answer before we run. Pick one, or enter your own.
How much better than its comparison a pocket must be		Needs an answer before we run. Pick one, or enter your own.
Is it real		
How sure a difference must be before it counts		Needs an answer before we run. Pick one, or enter your own.
How often a real gap should be caught	80% (recommended)	0.8 The usual standard. Used only to say how big a gap each pocket's size could

Right: the shaded cells cover the Choose and Or enter your own columns. Grey cells marked n/a are rows where you can only pick from the list. They are not waiting for you.

Step 4: Fill in the shaded cells

For each shaded row, click the Choose cell and pick from the list. To use a number that isn't on the list, leave Choose alone and **type the number in the next column, Or enter your own**. Your number wins.

On this walk:

Row	Setting	Answer
8	How old a loan must be to count	Every loan (picked)
15	Fewest loans in a pocket before it is tested	30 (picked)
16	Fewest loans with a loss before a loss rate is tested	10 (picked)
18	Smallest excess loss worth reporting	1% of the book's total losses (picked)
20	What a pocket is judged against	The rest of its band (picked)
21	How much worse than its comparison a pocket must be	1.4 typed in Or enter your own
22	How much better than its comparison a pocket must be	0.8 times (picked)
24	How sure a difference must be before it counts	95% (picked)

Save and close the workbook.

Control center					
Fill in the shaded cells before running. Those are the calls we make on every job: what's material, how many loans is enough to matter, and how much worse than its peers a pocket has to be. Everything else starts on a reasonable setting, so change it only if the population calls for it. Pick from the list, or type your own number in the next column and that wins. If a row says re-run, run the script again after changing it.					
Shaded = still needs an answer					
Setting	Choose	Or enter your own use	What it means	Takes effect	key
What the cube looks at					
Which cuts to build	Every column alone, and every band crossed with every dimension (recommended)	n/a	pairs	Finds the most. Ten bands and eight dimensions make 80 grids, so the Where re-run to look tab ranks every pocket and you read only the top.	grids
How far to drill into a flagged pocket	One level - cut each flagged pocket by every other column (recommended)	n/a		1 Shows which third factor explains the bleed inside the pocket. Most of the new pockets will be small.	re-run drill_depth
How old a loan must be to count	Every loan			0 New loans have not had time to go bad, so a pocket full of recent loans looks better than it is.	re-run min_age_months
How columns are recognised					
A number column with this many values or fewer is a category	12 (recommended)			12 Term (36, 48, 60) or a grade from 1 to 10 is read as categories, not cut into bands.	re-run few_values
A text column with more values than this is too fine to cut by	50 (recommended)			50 State (50 or so) is still a dimension; a city or a dealer name is raised as a question instead.	re-run many_values
How many bands a number column is cut into	5 (recommended)			5 Fine enough to see a trend, with each band holding a fifth of the loans.	re-run band_count
Where the band edges fall	Each band holds about the same number of loans (recommended)	n/a	equal_loans	Every band can be tested equally well. Edges land on odd numbers like 663.	re-run band_cut
Is it enough loans to matter					
Fewest loans in a pocket before it is tested	30			30 The old workbook's value. Stops a test on a handful of loans; the test itself decides whether a gap is luck.	live min_loans
Fewest loans with a loss before a loss rate is tested	10			10 A loss rate rests on how many loans lost money, not how many a pocket holds. Ten is a common floor.	live min_events
Is it material					
Smallest excess loss worth reporting	1% of the book's total losses		1% of losses	Scales with the book, so the same setting works on any population.	live materiality
Does it underperform					
What a pocket is judged against	The rest of its band	n/a	peers	Broker under 620 against the rest of under 620. A pocket is never compared with itself.	live compare_to
How much worse than its comparison a pocket must be			1.4	1.4 Your own value, in place of the options (a multiple above 1, such as 1.4).	live worse_at
How much better than its comparison a pocket must be				0.8 The old workbook's value.	live better_at
Is it real					
How sure a difference must be before it counts	95%			0.95 The usual standard. About 1 in 20.	live confidence
How often a real gap should be caught	80% (recommended)			0.8 The usual standard. Used only to say how big a gap each pocket's size could show; it never flags anything.	live power
Allowing for testing many pockets at once	Hold down the share of lucky finds (Benjamin-Hochberg) (recommended)	n/a	bh	Of the pockets that pass, about 5% at 95% are expected to be luck. Suits scanning for leads.	live many_tests
Proving it					
Where a flagged pocket is tested	Found on earlier originations, confirmed on later ones	n/a	time	The strongest evidence for the LOB: the pattern held on loans it was not	re-run proof

	(recommended)				
Is it enough loans to matter					
Fewest loans in a pocket before it is tested	30			30 The old workbook's value. Stops a test on a handful of loans; the test itself decides whether a gap is luck.	
Fewest loans with a loss before a loss rate is tested	10			10 A loss rate rests on how many loans lost money, not how many a pocket holds. Ten is a common floor.	
Is it material					
Smallest excess loss worth reporting	1% of the book's total losses		1% of losses	Scales with the book, so the same setting works on any population.	
Does it underperform					
What a pocket is judged against	The rest of its band	n/a	peers	Broker under 620 against the rest of under 620. A pocket is never compared with itself.	
How much worse than its comparison a pocket must be			1.4	1.4 Your own value, in place of the options (a multiple above 1, such as 1.4).	
How much better than its comparison a pocket must be	0.8 times			0.8 The old workbook's value.	
Is it real					
How sure a difference must be before it counts	95%			0.95 The usual standard. About 1 in 20.	

Right: no shading left in the setting rows. In use shows the value that will be used (1.4 for row 21, your own number). What it means explains the choice.

Watch for:

- **Don't type your own number into the Choose cell.** Excel lets you, and In use then shows #N/A. Delete it and type it one cell to the right. (See "When the tool refuses" below.)
- **Type shares as shares.** Confidence is 0.95, not 95. A multiple is 1.4, not 1.4x. The tab doesn't warn you. Step 9 does.
- **Not all your answers are used yet.** On this build, loan age, fewest loans with a loss, materiality, what a pocket is judged against and the many-tests allowance are asked for but not applied. The run shows every comparison and a materiality table, and the call is yours when you read it (step 11).

Step 5: Check your answers read back

```
cube control --read control.xlsx
```

```
terminal - origination-cube
```

```
$ cube control --read control.xlsx
```

```
grids          pairs
drill_depth    1
min_age_months 0
few_values     12
many_values    50
band_count     5
band_cut       equal_loans
min_loans      30
min_events     10
materiality    1% of losses
compare_to     peers
worse_at       1.4
better_at      0.8
confidence     0.95
power          0.8
many_tests     bh
proof          time
(exit code 0)
```

Right: one line per setting, no REFUSED. The names are the tool's short names: worse_at 1.4 is row 21, confidence 0.95 is row 24, compare_to peers is "the rest of its band".

If a shaded cell was missed, it says so and names the cell:

```
terminal - origination-cube
```

```
$ cube control --read control-blank.xlsx
```

```
REFUSED: 8 setting(s) to fix:
```

```
- Control!C8: `How old a loan must be to count` needs an answer. Pick one, or enter your own in column D
- Control!C15: `Fewest loans in a pocket before it is tested` needs an answer. Pick one, or enter your own in column D
- Control!C16: `Fewest loans with a loss before a loss rate is tested` needs an answer. Pick one, or enter your own in column D
- Control!C18: `Smallest excess loss worth reporting` needs an answer. Pick one, or enter your own in column D
- Control!C20: `What a pocket is judged against` needs an answer. Pick one, or enter your own in column D
- Control!C21: `How much worse than its comparison a pocket must be` needs an answer. Pick one, or enter your own in column D
- Control!C22: `How much better than its comparison a pocket must be` needs an answer. Pick one, or enter your own in column D
- Control!C24: `How sure a difference must be before it counts` needs an answer. Pick one, or enter your own in column D
(exit code 2)
```

Step 6: Write the cube file

```
cube init demo/loans.csv -o cube.yaml --control control.xlsx
```

```
terminal - origination-cube
```

```
$ cube init demo/loans.csv -o cube.yaml --control control.xlsx
```

```
wrote cube.yaml
band      FICO, ORIG_BAL, GCO_AMT, RANR_AMT
dimension CHANNEL, BAD_FLAG
key       LOAN_NBR
2 odd value pattern(s) raised as questions; each is used as recorded until answered
Next: answer every [CONFIRM: ...] in the file, then `cube validate`.
(exit code 0)
```

Right: wrote cube.yaml, then how each column was sorted (band, dimension, key) and how many odd values were raised as questions.

The last line says to answer every [CONFIRM: ...]. With a filled Control tab there usually aren't any. **What does stop the run is columns_confirmed: no.** Go to step 7.

Step 7: Read the cube file

Open cube.yaml in any text editor. The highlighted lines are the ones you act on:

```
editor - cube.yaml (as cube init wrote it)
1 # Written by `cube init` from loans.csv on 2026-09-25.
2 # 2,000 rows, 7 columns. Every column is below, in its section
3 # or under 'Not cut by', with the reason. Move a line to change what a column is.
4 # Nothing marked [CONFIRM: ...] is decided for you; the run refuses until each is answered.
5 name: loans
6 schema_version: 1
7
8 # REQUIRED COLUMNS, suggested by cube init. Check each. If one is wrong, type the right
9 # column name over it. Then set columns_confirmed to yes; the run refuses until you do.
10 key: LOAN_NBR           # SUGGESTED - name contains 'loan'; a different value on every row. Needed: loan or application number, to map
    results back to loans
11 booked: ORIG_BAL       # SUGGESTED - name contains 'bal'; positive on every loan. Needed: the booked or loan amount that weights the
    reporting-level rates
12 outcome: BAD_FLAG      # SUGGESTED - name contains 'bad'; 0 and 1 (4.8% are 1); 1 other value(s), counted when read. Needed: a yes/no
    column (0/1); or write {field: COLUMN, is: VALUE}
13 gco: GCO_AMT           # SUGGESTED - name contains 'gco'; zero on 95% of loans, never negative. Needed: gross charge-off dollars
14 ranr: RANR_AMT         # SUGGESTED - name contains 'ranr'; numbers. Needed: RANR dollars (signed values are kept)
15 columns_confirmed: no
16
17 missing: {}            # answer the questions at the bottom to add rules here
18
19 # CHECK the bands and dimensions: take out any column recorded AFTER the loan was made
20 # (a charge-off flag, a status, days past due). Cutting by an outcome is circular, and
21 # the tool cannot tell when a column was recorded. GCO and RANR are left out for you
22 # once they are named under measures.
23 bands:                 # cut into 5, equal loans
24   - {name: fico, field: FICO, count: 5, cut: equal_loans} # numbers with 277 values
25   - {name: orig_bal, field: ORIG_BAL, count: 5, cut: equal_loans} # numbers with 1,998 values
26   - {name: gco_amt, field: GCO_AMT, count: 5, cut: equal_loans} # numbers with 97 values; 1 value(s) not a number, counted when read
27   - {name: ranr_amt, field: RANR_AMT, count: 5, cut: equal_loans} # numbers with 1,963 values
28
29 dimensions:
30   - {name: channel, field: CHANNEL} # text with 3 values
31   - {name: bad_flag, field: BAD_FLAG} # numbers with only 3 values: read as categories
32
33 # Not cut by:
34 #   LOAN_NBR           key: a different value on every row
35
36 # The core rates (outcome by loans and by booked dollars, GCO and RANR per booked dollar)
37 # are built from the required columns on every run. Extras go here; per: each_loan
38 # divides by the number of loans instead of a column (a straight average).
39 measures:
40   - {name: loans, mode: count}
41
42 benchmark:
43   min_units: 30
44   worse_at: 1.4
45   better_at: 0.8
46   confidence: 0.95
47   power: 0.8
48
49 # Odd values. Each is used AS RECORDED until you answer: real, or missing.
50 # Nothing stops the run; every output lists the questions still open.
51 questions:
52   - {column: FICO, pattern: repeated_value, value: -9999, rows: 40, answer: }
53   - {column: RANR_AMT, pattern: negatives, rows: 707, answer: }
```

Check, in order:

1. **Lines 10 to 14, the five required columns.** Each shows a suggestion and its reason. Is key the loan number? Is booked the loan amount? Is outcome the charge-off or bad flag? Are gco and ranr the dollar columns? If one is wrong, type the right column name over it.
2. **Line 12, the outcome.** Read the reason. Here it says 0 and 1 (4.8% are 1); 1 other value(s). One loan has a value that isn't 0 or 1. The run leaves it out and counts it (step 10). Ask the bank what it means.
3. **Lines 23 to 31, bands and dimensions.** Take out any column recorded **after** the loan was made: a charge-off flag, a status, days past due. Cutting the book by its own outcome proves nothing. Here that is bad_flag on line 31. gco_amt and ranr_amt (lines 26 and 27) are dropped by the run itself, with a warning. You can leave them.
4. **Lines 52 and 53, odd values.** -9999 in FICO on 40 loans is the bureau's "no score" code: answer missing. Negative RANR on 707 loans is real: answer real.

Step 8: Make the edits

- Line 15: `columns_confirmed: no` → `columns_confirmed: yes`
- Delete line 31 (`bad_flag`)
- Line 52: `answer: }` → `answer: missing}`
- Line 53: `answer: }` → `answer: real}`

Save.

```
editor - cube.yaml (after your edits: line 15, line 31 removed, lines 51-52)
1 # Written by `cube init` from loans.csv on 2026-09-25.
2 # 2,000 rows, 7 columns. Every column is below, in its section
3 # or under 'Not cut by', with the reason. Move a line to change what a column is.
4 # Nothing marked [CONFIRM: ...] is decided for you; the run refuses until each is answered.
5 name: loans
6 schema_version: 1
7
8 # REQUIRED COLUMNS, suggested by cube init. Check each. If one is wrong, type the right
9 # column name over it. Then set columns_confirmed to yes; the run refuses until you do.
10 key: LOAN_NBR           # SUGGESTED - name contains 'loan'; a different value on every row. Needed: loan or application number, to map
    results back to loans
11 booked: ORIG_BAL       # SUGGESTED - name contains 'bal'; positive on every loan. Needed: the booked or loan amount that weights the
    reporting-level rates
12 outcome: BAD_FLAG      # SUGGESTED - name contains 'bad'; 0 and 1 (4.8% are 1); 1 other value(s), counted when read. Needed: a yes/no
    column (0/1); or write {field: COLUMN, is: VALUE}
13 gco: GCO_AMT           # SUGGESTED - name contains 'gco'; zero on 95% of loans, never negative. Needed: gross charge-off dollars
14 ranr: RANR_AMT         # SUGGESTED - name contains 'ranr'; numbers. Needed: RANR dollars (signed values are kept)
15 columns_confirmed: yes
16
17 missing: {}            # answer the questions at the bottom to add rules here
18
19 # CHECK the bands and dimensions: take out any column recorded AFTER the loan was made
20 # (a charge-off flag, a status, days past due). Cutting by an outcome is circular, and
21 # the tool cannot tell when a column was recorded. GCO and RANR are left out for you
22 # once they are named under measures.
23 bands:                 # cut into 5, equal loans
24   - {name: fico, field: FICO, count: 5, cut: equal_loans} # numbers with 277 values
25   - {name: orig_bal, field: ORIG_BAL, count: 5, cut: equal_loans} # numbers with 1,998 values
26   - {name: gco_amt, field: GCO_AMT, count: 5, cut: equal_loans} # numbers with 97 values; 1 value(s) not a number, counted when read
27   - {name: ranr_amt, field: RANR_AMT, count: 5, cut: equal_loans} # numbers with 1,963 values
28
29 dimensions:
30   - {name: channel, field: CHANNEL} # text with 3 values
31
32 # Not cut by:
33 #   LOAN_NBR           key: a different value on every row
34
35 # The core rates (outcome by loans and by booked dollars, GCO and RANR per booked dollar)
36 # are built from the required columns on every run. Extras go here; per: each_loan
37 # divides by the number of loans instead of a column (a straight average).
38 measures:
39   - {name: loans, mode: count}
40
41 benchmark:
42   min_units: 30
43   worse_at: 1.4
44   better_at: 0.8
45   confidence: 0.95
46   power: 0.8
47
48 # Odd values. Each is used AS RECORDED until you answer: real, or missing.
49 # Nothing stops the run; every output lists the questions still open.
50 questions:
51   - {column: FICO, pattern: repeated_value, value: -9999, rows: 40, answer: missing}
52   - {column: RANR_AMT, pattern: negatives, rows: 707, answer: real}
```

Step 9: Validate

```
cube validate cube.yaml --data demo/loans.csv
```

```
terminal - origination-cube
$ cube validate cube.yaml --data demo/loans.csv
accepted: loans over 2,000 rows, 2 grid(s), 44 tie-out checks agree
WARNING: `GCO_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
WARNING: `RANR_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
(exit code 0)
```

Right: accepted, the number of grids (2 here: FICO by channel, loan amount by channel), and every tie-out check agrees. The two warnings about GCO_AMT and RANR_AMT are expected (step 7, point 3).

If it says REFUSED, it lists each problem. Fix them and run it again. See "When the tool refuses" below.

Step 10: Run it

```
cube run cube.yaml --data demo/loans.csv
```

The top of the output:

```
terminal - origination-cube
$ cube run cube.yaml --data demo/loans.csv
Cube: loans (demo/loans.csv, 2,000 rows)
Tie-out: 44 of 44 checks agree (every grid adds up to the book; medians are not added up, by design)

WARNINGS
- `GCO_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
- `RANR_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome

Left out of a figure, and counted rather than read as zero
outcome_loans: 1 rows (1 x BAD_FLAG flag not 0 or 1)
outcome_booked: 2 rows (1 x BAD_FLAG flag not 0 or 1; 1 x ORIG_BAL blank)
gco_rate: 2 rows (1 x GCO_AMT not a number; 1 x ORIG_BAL blank)
ranr_rate: 1 rows (1 x ORIG_BAL blank)

Topline (the whole book)
outcome_loans    4.80%    COUNT(loans where BAD_FLAG = 1) / COUNT(loans)    (1,999 loans)
outcome_booked   4.66%    SUM(ORIG_BAL where BAD_FLAG = 1) / SUM(ORIG_BAL)    (1,998 loans)
gco_rate         2.58%    SUM(GCO_AMT) / SUM(ORIG_BAL)    (1,998 loans)
ranr_rate        0.28%    SUM(RANR_AMT) / SUM(ORIG_BAL)    (1,999 loans)
loans            2,000    Loans (rows)

Band edges used
fico: 653, 684, 714, 745.4
orig_bal: 15327.1, 26463.9, 39152.5, 49789.1

What this book can show (evidence for your settings, not a setting)
outcome_loans: about 1,083 loans before a 1.4x gap can be told from luck 80% of the time at 95% confidence (54.2% of
  this book's loans)
outcome_booked: about 1,393 loans before a 1.4x gap can be told from luck 80% of the time at 95% confidence (69.7%
  of this book's loans)
gco_rate: about 1,459 loans before a 1.4x gap can be told from luck 80% of the time at 95% confidence (73.0% of this
  book's loans)
ranr_rate: about 212 loans before a 1.4x gap can be told from luck 80% of the time at 95% confidence (10.6% of this
  book's loans)
A pocket smaller than that can still show a bigger gap; each pocket below says the
smallest gap its size could show.
```

Read, in order:

1. **Tie-out:** every check agrees. If any doesn't, stop. The grids don't add up to the book.
2. **Left out of a figure:** loans missing from a rate, and why. Here it's the one loan with outcome 2, one #N/A in GCO and one blank loan amount. Small, and named.
3. **Topline:** the whole book's rates. These are what every pocket is compared with.
4. **Band edges used:** where the score and amount bands were cut. By default each band holds a fifth of the loans, so the edges land on odd numbers (653, 745.4). Step 12 shows how to use the LOB's own cut points.
5. **What this book can show:** how many loans a pocket needs before a 1.4x gap shows reliably. That is evidence for your settings. It isn't applied.

Step 11: Read the bleed list

Each grid has a table, then **Where it bleeds**: the pockets with the most excess over the book's rate, largest first.

```
terminal - origination-cube
```

Grid fico x channel - outcome_loans: COUNT(loans where BAD_FLAG = 1) / COUNT(loans)								topline 4.80%, median of cells 3.68%
each cell: rate, then rate over topline								
		Branch		Broker		Online		All
under 653	4.62%	0.96x	17.69%	3.68x	6.35%	1.32x	9.59%	2.00x
653 to under 684	2.90%	0.60x	8.66%	1.80x	7.14%	1.49x	6.14%	1.28x
684 to under 714	1.60%	0.33x	3.68%	0.77x	1.49%	0.31x	2.28%	0.47x
714 to under 745.4	5.34%	1.11x	4.69%	0.98x	2.22%	0.46x	4.06%	0.85x
745.4 and over	1.48%	0.31x	3.25%	0.68x	2.24%	0.47x	2.30%	0.48x
(blank)					0.00%	0.00x	0.00%	0.00x
(missing by rule)	0.00%	0.00x	5.56%	1.16x	0.00%	0.00x	2.50%	0.52x
All	3.14%	0.65x	7.55%	1.57x	3.74%	0.78x	4.80%	1.00x

Where it bleeds: excess loans where BAD_FLAG = 1 over the topline rate, largest first

under 653 / Broker: 17 over, 130 loans
share of losses / share of volume 3.68x
vs rest of book 4.53x (p <0.0001): worse than benchmark
vs rest of band 3.24x (p 0.0008): worse than benchmark
this many loans can show a gap of 2.26x or more
653 to under 684 / Broker: 5 over, 127 loans
share of losses / share of volume 1.80x
vs rest of book 1.91x (p 0.1063): gap could be luck
vs rest of band 1.76x (p 0.1880): gap could be luck
this many loans can show a gap of 2.28x or more
653 to under 684 / Online: 3 over, 126 loans
share of losses / share of volume 1.49x
vs rest of book 1.54x (p 0.2887): gap could be luck
vs rest of band 1.26x (p 0.5840): in line
this many loans can show a gap of 2.28x or more
under 653 / Online: 2 over, 126 loans
share of losses / share of volume 1.32x
vs rest of book 1.35x (p 0.4602): in line
vs rest of band 0.57x (p 0.1010): gap could be luck
this many loans can show a gap of 2.28x or more
714 to under 745.4 / Branch: 1 over, 131 loans
share of losses / share of volume 1.11x
vs rest of book 1.12x (p 0.7758): in line
vs rest of band 1.56x (p 0.3973): gap could be luck
this many loans can show a gap of 2.25x or more

Materiality evidence: what each level would keep (the level is your call)
0.5% of book losses (0): 5 pocket(s), 100% of this grid's excess

For each pocket:

- **share of losses / share of volume:** how much worse than the book it runs. 3.68x means nearly four times its share.
- **vs rest of book** and **vs rest of band:** the pocket against everything else, and against the rest of its score band. Each has a p-value and a word:
 - worse than benchmark: past your 1.4x and unlikely to be luck at 95%
 - gap could be luck: past 1.4x (or below 0.8x) but not proven
 - in line: inside those limits
 - too few loans to test: under your 30
- **this many loans can show a gap of...:** the smallest gap a pocket this size could prove.

Right on this book: the top pocket is **under 653 / Broker**. It runs 3.7x the book on bad-loan share, 3.4x on GCO dollars, and is worse than benchmark against both the rest of the book and the rest of its band. That is the planted pocket. The within-band line shows it's Broker, not the score band: Branch and Online under 653 aren't worse than their band.

Materiality is under each list: what each floor (0.5% to 10% of book losses) would keep. The Control tab asked for a floor, but this build doesn't draw the line. Pick the floor as you read.

Take care with the RANR grids. The list ranks the highest RANR per booked dollar as "bleeding". Whether a higher RANR is worse depends on what the bank's RANR column holds. Check that before you quote a RANR pocket.

Step 12 (when you know the LOB's cut points): use your own band edges

The default five bands put the first score edge at 653, which dilutes a problem that sits below 620. If the LOB already cuts at 620, 680 and 740, change the FICO line in cube.yaml:

```
- {name: fico, field: FICO, edges: [620, 680, 740]}
```

Then run it again:

```
terminal - origination-cube
$ cube run cube-620.yaml --data demo/loans.csv
...
Grid fico x channel - gco_rate: SUM(GCO_AMT) / SUM(ORIG_BAL) topline 2.58%, median of cells 2.26%
each cell: rate, then rate over topline
```

		Branch		Broker		Online		All
under 620	3.41%	1.32x	16.34%	6.33x	3.37%	1.31x	7.69%	2.98x
620 to under 680	2.17%	0.84x	5.03%	1.95x	3.72%	1.44x	3.65%	1.41x
680 to under 740	1.46%	0.56x	2.34%	0.91x	0.93%	0.36x	1.55%	0.60x
740 and over	1.60%	0.62x	0.99%	0.38x	1.29%	0.50x	1.29%	0.50x
(blank)					0.00%	0.00x	0.00%	0.00x
(missing by rule)	0.00%	0.00x	5.54%	2.14x	0.00%	0.00x	2.93%	1.13x
All	1.82%	0.70x	4.00%	1.55x	1.95%	0.75x	2.58%	1.00x

```
Where it bleeds: excess GCO_AMT over the topline rate, largest first
under 620 / Broker: 234,021 over, 53 loans
share of losses / share of volume 6.33x
vs rest of book 7.38x (p 0.0002): worse than benchmark
vs rest of band 4.82x (p 0.0017): worse than benchmark
this many loans can show a gap of 3.51x or more
620 to under 680 / Broker: 148,468 over, 183 loans
share of losses / share of volume 1.95x
vs rest of book 2.16x (p 0.0533): gap could be luck
vs rest of band 1.70x (p 0.1847): gap could be luck
```

Right: under 620 / Broker is now at the top at 6.3x the book on GCO dollars, 234,021 over, on 53 loans. That is the planted pocket at its true size.

When the tool refuses

These are the wrong turns tried on this walk and what the tool said. None of them loses work. Fix the cell or line it names and run the step again.

A. Validating before confirming the columns (skipping step 8):

```
terminal - origination-cube

$ cube validate cube.yaml --data demo/loans.csv
REFUSED: cube.yaml has 1 problem(s):
- the required columns were suggested by `cube init` and are not confirmed yet (key: LOAN_NBR, booked: ORIG_BAL, outcome: BAD_FLAG, gco: GCO_AMT, ranr: RANR_AMT). Check each; type the right column name over any that is wrong; then set `columns_confirmed: yes`
(exit code 2)
```

Set columns_confirmed: yes after checking lines 10 to 14.

B. Leaving a shaded cell empty:

```
terminal - origination-cube

$ cube init demo/loans.csv -o wt-empty.yaml --control wt-empty.xlsx
REFUSED: wt-empty.xlsx has 1 setting(s) to answer:
- Control!C24: `How sure a difference must be before it counts` needs an answer. Pick one, or enter your own in column D
(exit code 2)
```

It names the cell (Control!C24). Answer it and save.

C. Typing text where a number goes (1.4x in row 21):

```
terminal - origination-cube

$ cube init demo/loans.csv -o wt-text.yaml --control wt-text.xlsx
REFUSED: wt-text.xlsx has 1 setting(s) to answer:
- Control!D21: `How much worse than its comparison a pocket must be` needs a multiple above 1, such as 1.4; got '1.4x'
(exit code 2)
```

The tab itself shows nothing wrong, so the refusal only comes here:

Setting	Choose	Or enter your own	In use	What
How much worse than its comparison a pocket must be		1.4x	1.4x	Your
it means				
own value, in place of the options (a multiple above 1, such as 1.4).		Takes effect	live	key worse_at

Type 1.4.

D. Typing your own number into the Choose cell (1.4 in C21):

Setting	Choose	Or enter your own	In use	What
How much worse than its comparison a pocket must be	1.4			#N/A
it means				
		Takes effect	#N/A live	key worse_at

```
terminal - origination-cube

$ cube init demo/loans.csv -o wt-intochoose.yaml --control wt-intochoose.xlsx
REFUSED: wt-intochoose.xlsx has 1 setting(s) to answer:
- Control!C21: '1.4' is not an option for `How much worse than its comparison a pocket must be`
(exit code 2)
```

Delete it from Choose and type it in Or enter your own.

E. Typing 95 for 95% confidence (in D24). Neither the tab, cube control --read nor cube init catches it:

```

terminal - origination-cube

$ cube init demo/loans.csv -o wt-95.yaml --control wt-95.xlsx
wrote wt-95.yaml
band      FICO, ORIG_BAL, GCO_AMT, RANR_AMT
dimension CHANNEL, BAD_FLAG
key       LOAN_NBR
2 odd value pattern(s) raised as questions; each is used as recorded until answered
Next: answer every [CONFIRM: ...] in the file, then `cube validate`.
(exit code 0)

```

cube validate does, but it names the line in cube.yaml, not the cell you typed in:

```

terminal - origination-cube

$ cube validate wt-95.yaml --data demo/loans.csv
REFUSED: wt-95.yaml has 1 problem(s):
- benchmark.confidence must be a share between 0.5 and 1, such as 0.95; got 95
(exit code 2)

```

Fix it on the Control tab (D24 → 0.95) and run step 6 again. If you fix only cube.yaml, the next cube init puts 95 back.

F. Leaving BAD_FLAG in as a dimension: no harm. The run drops it and warns:

```

terminal - origination-cube

$ cube validate wt-keepbad.yaml --data demo/loans.csv
accepted: loans over 2,000 rows, 2 grid(s), 44 tie-out checks agree
WARNING: `GCO_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
WARNING: `RANR_AMT` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
WARNING: `BAD_FLAG` is not cut by: it is the top of a rate, so cutting by it would cut the book by its own outcome
(exit code 0)

```